Available online at www.sciencedirect.com
SCIENCE DIRECT
Mathematical and Computer Modelling 41 (2005) 379–389
MATHEMATICAL AND COMPUTER MODELLING
www.elsevier.com/locate/mcm

Modeling Distributed Software Defect Removal Effectiveness in the Presence of Code Churn

LIANG TIAN AND A. NOORE*
Lane Department of Computer Science and Electrical Engineering
West Virginia University
Morgantown, WV 26506, U.S.A.
afzel.noore@mail.wvu.edu

(Received October 2003; revised and accepted October 2004)

Abstract—Two types of discrete defect removal models that consider the dynamics of code churn behavior during software testing phases under distributed software development environment are proposed. The first model is based on a sequential debugging process, while the second model is based on an iterative debugging process during each testing phase. The mathematical relationship between the number of defects detected during a testing phase and the total estimated remaining defects at the end of the same testing phase for both models are elaborated in detail. The defect detection ratio is identified to have the greatest contribution to the variance of the estimated number of remaining defects based on the sensitivity analysis using Monte-Carlo simulation. Using the proposed models, we quantitatively show how to estimate the number of defects by varying the defect detection ratio, defect correction ratio, the percentage of added code, and the percentage of deleted code. © 2005 Elsevier Ltd. All rights reserved.

Keywords—Defect removal, Distributed environment, Imperfect debugging.

1. INTRODUCTION

With increasing complexity of software, the quality of software is one of the ultimate goals in the software development process. The number of estimated remaining defects is an important measure of software quality. Several traditional discrete defect removal models describe the software defect detection and defect correction behavior during the software testing phases [1–7]. Remus and Zilles [1] first constructed a single cycle of detection-correction model to estimate the number of remaining defects at the end of a certain phase based on the number of defects detected in the same phase. Yu *et al.* [2] revised the Remus-Zilles model so that the debugging process is iteratively checked before proceeding to the next phase. These two models are the basis of currently existing phase-based software defect removal models. One of the implicit assumptions in these models is that the code size remains unchanged during the testing process. However, large-scale and complex software projects are usually developed in a distributed environment. Also, during software development, the size of the code changes as requirements, design constraints

*Author to whom all correspondance should be addressed.

0895-7177/05/$ - see front matter © 2005 Elsevier Ltd. All rights reserved. Typeset by $\mathcal{A}\mathcal{M}\mathcal{S}$-TEX
doi:10.1016/j.mcm.2004.10.021

or budget change [8–10]. Each of the distributed software development location has its own testing process, including several testing phases and corresponding code churn information, such as added, modified, and deleted code. In this paper, we propose to model the dynamic behavior of code churn in software testing phases under a distributed software development environment. Two types of defect removal models are derived based on whether the debugging process is sequential or iterative in each testing phase. The mathematical relationship between the number of defects detected during the jth testing phase in different distributed development locations and the total estimated remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase for both the sequential and iterative debugging models are derived. Sensitivity analysis using Monte-Carlo simulation is used to identify the effect of estimation parameters that contribute most to the variance of the estimated number of remaining defects. Further, based on empirical data analysis, comparison of defect removal effectiveness between the proposed two models and the existing approaches are explored. The following sections present the development of two models incorporating code churn information during the software debugging process in a distributed environment.

2. TYPE-I SEQUENTIAL DEBUGGING MODEL

The proposed Type-I discrete defect removal model is developed under the assumption that the debugging process in each testing phase is sequential. The detailed process for Type-I sequential debugging model is illustrated in Figure 1. The assumptions, symbols, and notations for this model are described below.

Each symbol has two subscripts. The first subscript refers to the software development location index i, and the second subscript refers to the testing phase index j in a specific development location.

- N denotes the number of development locations in the distributed software development environment
- S_{ij} original software code size in thousand lines of code (KLOC) or in function point (FP) at the beginning of the jth testing phase in the ith development location
- $A_{i(j+1)}$ added and modified code in KLOC or FP at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location
- $R_{i(j+1)}$ deleted and modified code from the original software (S_{ij}) in KLOC or FP at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location
- D_{ij} original number of remaining defects at the beginning of the jth testing phase in the ith development location
- DF_{ij} original number of defects detected during the jth testing phase in the ith development location

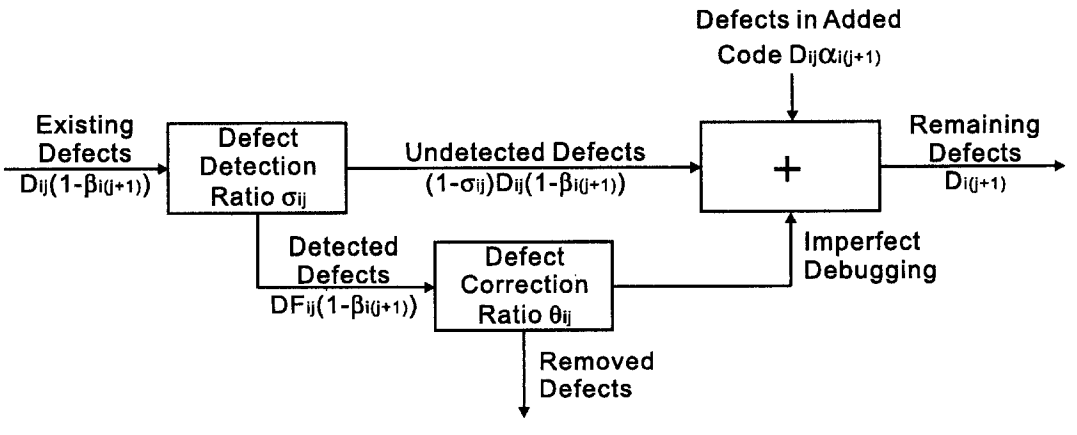

Figure 1. Type-I sequential debugging model.

$D_{i(j+1)}$ number of remaining defects at the end of the jth testing phase, or the number of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location

$\alpha_{i(j+1)}$ percentage of added and modified code compared to the original software code size

$\beta_{i(j+1)}$ percentage of deleted and modified code compared to the original software code size

σ_{ij} defect detection ratio in the jth testing phase in the ith development location

θ_{ij} defect correction ratio in the jth testing phase in the ith development location; thus, $(1-\theta_{ij})$ is the rate of imperfect debugging

$D_{ij}\beta_{i(j+1)}$ number of defects contained in the code that will be deleted and modified at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location; this code has already undergone debugging in the jth testing phase

$D_{ij}\alpha_{i(j+1)}$ number of defects contained in the code that will be added and modified at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location

The added and modified code $A_{i(j+1)}$, and the deleted and modified code $R_{i(j+1)}$ have the same defect density (defect/KLOC or defect/FP) as the original code S_{ij}. $R_{i(j+1)}$ is a part of S_{ij}, and has the same defect density. For $A_{i(j+1)}$, we assume that this part of the code is developed by the same development team as in S_{ij} under the same development environment. In fact, for a distributed software development environment, the defect density of the added and modified code $A_{i(j+1)}$ would vary from location to location depending on corporate history, experience of the development team, and the prior development effort of the software. Thus, the overall defect density is in effect nonhomogeneous and emulates a piecewise nonlinear function.

The defects contained in the deleted and modified code $R_{i(j+1)}$, the added and modified code $A_{i(j+1)}$, and the original code S_{ij} have the same probability of detection and/or correction.

The number of remaining defects at the end of the jth testing phase or the number of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location is given by

$$\begin{aligned}
D_{i(j+1)} &= D_{ij}\left(1-\beta_{i(j+1)}\right) - \theta_{ij}D_{ij}\left(1-\beta_{i(j+1)}\right)\sigma_{ij} + D_{ij}\alpha_{i(j+1)} \\
&= \frac{\mathrm{DF}_{ij}}{\sigma_{ij}}\left(1-\beta_{i(j+1)}\right) - \theta_{ij}\frac{\mathrm{DF}_{ij}}{\sigma_{ij}}\left(1-\beta_{i(j+1)}\right)\sigma_{ij} + \frac{\mathrm{DF}_{ij}}{\sigma_{ij}}\alpha_{i(j+1)} \\
&= \frac{\mathrm{DF}_{ij}}{\sigma_{ij}}\left(1-\beta_{i(j+1)}\right) - \theta_{ij}\,\mathrm{DF}_{ij}\left(1-\beta_{i(j+1)}\right) + \frac{\mathrm{DF}_{ij}}{\sigma_{ij}}\alpha_{i(j+1)} \qquad (1) \\
&= \mathrm{DF}_{ij}\left(1-\beta_{i(j+1)}\right)\left(\frac{1}{\sigma_{ij}} - \theta_{ij}\right) + \frac{\mathrm{DF}_{ij}}{\sigma_{ij}}\alpha_{i(j+1)} \\
&= \left[\left(1-\beta_{i(j+1)}\right)\left(\frac{1}{\sigma_{ij}} - \theta_{ij}\right) + \frac{\alpha_{i(j+1)}}{\sigma_{ij}}\right] \times \mathrm{DF}_{ij}\,.
\end{aligned}$$

Let $Q_{ij} = (1-\beta_{i(j+1)})(1/\sigma_{ij} - \theta_{ij}) + \alpha_{i(j+1)}/\sigma_{ij}$, then we can denote the relationship between $D_{i(j+1)}$ and DF_{ij} as

$$D_{i(j+1)} = Q_{ij} \times \mathrm{DF}_{ij}\,. \qquad (2)$$

Here, DF_{ij} is the number of the defects detected in the jth testing phase of the ith development location, which can be easily observed. Once we estimate the value for Q_{ij}, we can estimate the number of defects remaining at the end of the jth testing phase or the number of defects remaining at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location.

The total number of estimated remaining defects (D) in a distributed environment containing N software development locations is

$$D = \sum_{i=1}^{N} D_{i(j+1)} = \sum_{i=1}^{N} Q_{ij} \times \mathrm{DF}_{ij}, \qquad (3)$$

where j represents the most recently completed testing phase corresponding to each development location.

3. TYPE-II ITERATIVE DEBUGGING MODEL

The proposed Type-II discrete defect removal model is shown in Figure 2 and is developed under the assumption that the debugging processes in each testing phase is iterative. In this model, the software is subjected to repeated debugging before proceeding to the next testing phase. The Type-II debugging model with two cycles of iteration is shown in Figure 3.

Most of the related assumptions, symbols, and notations in Type-II model are same as stated in Type-I model. To avoid redundancy, only the new or modified assumptions, symbols, and notations are explained.

DF_{ijk} original number of defects detected during the kth iteration in the jth testing phase of the ith development location

σ_{ijk} defect detection ratio in the kth iteration of the jth testing phase in the ith development location

In practice, the detection ratio is proportional to the testing time and usually decreases after each iteration of testing. So, $\sigma_{ij(k+1)}$ would be less than or equal to σ_{ijk}.

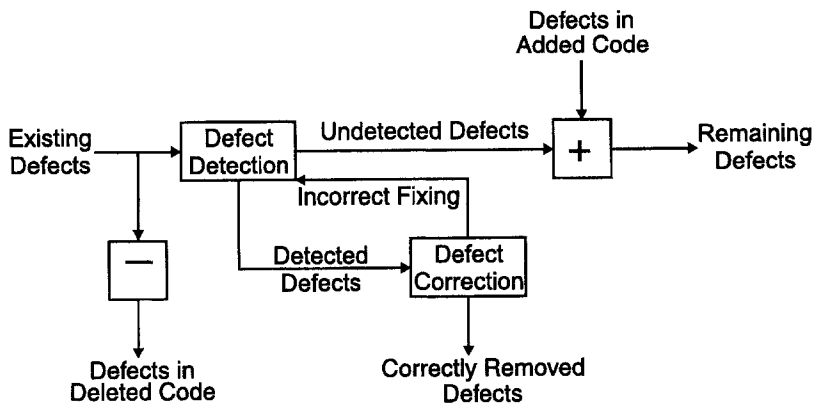

Figure 2. Type-II iterative debugging model.

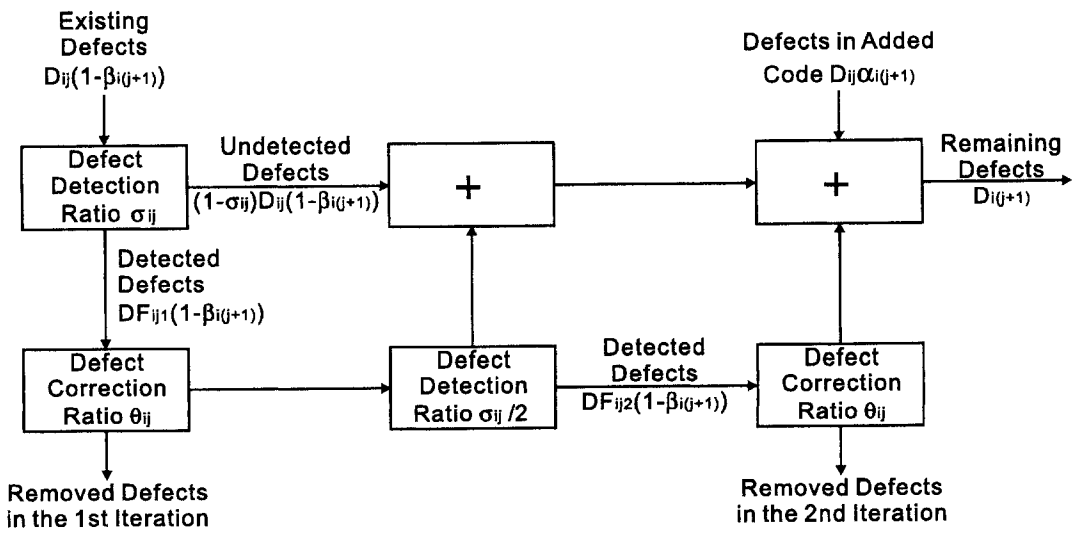

Figure 3. Type-II iterative debugging model with two cycles of iterations.

K defect detection ratio reduction factor; the defect detection ratio in the $(k+1)^{\text{th}}$ iteration of testing is reduced by factor K compared to the defect detection ratio in the kth iteration of testing. $\sigma_{ij(k+1)}$ is denoted by σ_{ijk} divided by K

θ_{ijk} defect correction ratio in the kth iteration of the jth testing phase in the ith development location

The correction ratio is always the same during a specific testing phase. So, we denote that $\theta_{ij1} = \theta_{ij2} = \cdots = \theta_{ijk} = \theta_{ij}$.

If the number of iterations, k, is two, and we denote $\sigma_{ij1} = \sigma_{ij}$, then we have

$$\sigma_{ij2} = \frac{\sigma_{ij}}{K}.$$

The number of defects detected in each testing phase is the sum of the defects detected in the first two iterations

$$\mathrm{DF}_{ij} = \mathrm{DF}_{ij1} + \mathrm{DF}_{ij2}, \tag{4}$$

$$\mathrm{DF}_{ij} = \sigma_{ij} D_{ij} + (1-\theta_{ij}) \frac{\sigma_{ij}^2}{K} D_{ij}. \tag{5}$$

Let $Q_{ija} = \sigma_{ij} + (1-\theta_{ij})(\sigma_{ij}^2/K)$, then we can denote the relationship between DF_{ij} and D_{ij} as

$$\mathrm{DF}_{ij} = Q_{ija} D_{ij}. \tag{6}$$

As before, the number of remaining defects at the end of the jth testing phase or the number of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location is given by

$$\begin{aligned} D_{i(j+1)} &= (1-\sigma_{ij}) D_{ij} \left(1-\beta_{i(j+1)}\right) + (1-\theta_{ij}) \left(1-\frac{\sigma_{ij}}{K}\right) \mathrm{DF}_{ij1} \left(1-\beta_{i(j+1)}\right) \\ &\quad + (1-\theta_{ij}) \mathrm{DF}_{ij2} \left(1-\beta_{i(j+1)}\right) + D_{ij}\alpha_{i(j+1)} \\ &= (1-\sigma_{ij}) D_{ij} \left(1-\beta_{i(j+1)}\right) + (1-\theta_{ij}) \left(1-\frac{\sigma_{ij}}{K}\right) D_{ij}\sigma_{ij} \left(1-\beta_{i(j+1)}\right) \\ &\quad + (1-\theta_{ij})^2 \frac{\sigma_{ij}^2}{K} D_{ij} \left(1-\beta_{i(j+1)}\right) + D_{ij}\alpha_{i(j+1)} \\ &= Q_{ijb} \times D_{ij}, \end{aligned} \tag{7}$$

where

$$Q_{ijb} = \left(1-\beta_{i(j+1)}\right) \left[(1-\sigma_{ij}) + (1-\theta_{ij}) \left(1-\frac{\sigma_{ij}}{K}\right) \sigma_{ij} + (1-\theta_{ij})^2 \frac{\sigma_{ij}^2}{K}\right] + \alpha_{i(j+1)}.$$

We denote the relationship between $D_{i(j+1)}$ and D_{ij} as

$$D_{i(j+1)} = Q_{ijb} D_{ij} \tag{8}$$

and

$$D_{i+1} = \frac{Q_{ijb}}{Q_{ija}} \times \mathrm{DF}_{ij}. \tag{9}$$

Let $Q'_{ij} = Q_{ijb}/Q_{ija}$, then the simplified equation is

$$D_{i(j+1)} = Q'_{ij} \times \mathrm{DF}_{ij}. \tag{10}$$

For quantitative purposes, we assume that the number of defects detected is reduced by at least half after each iteration in a testing phase.

When $K = 2$, we have

$$Q_{ija} = \sigma_{ij} + (1 - \theta_{ij}) \frac{\sigma_{ij}^2}{2}, \tag{11}$$

$$Q_{ijb} = \left(1 - \beta_{i(j+1)}\right) \left[(1 - \sigma_{ij}) + (1 - \theta_{ij}) \left(1 - \frac{\sigma_{ij}}{2}\right) \sigma_{ij} + (1 - \theta_{ij})^2 \frac{\sigma_{ij}^2}{2} \right] + \alpha_{i(j+1)}. \tag{12}$$

We use equation (10) to estimate the number of remaining defects. DF_{ij} is the sum of the defects detected during the first two iterations in the jth testing phase of ith development location, which can be easily observed. Once we find the value for Q'_{ij}, we can estimate the number of defects remaining at the end of jth testing phase or the number of defects remaining at the beginning of $(j+1)^{\text{th}}$ testing phase in the ith development location.

For Type-II iterative debugging model, the total number of estimated remaining defects (D) in a distributed environment comprising of N software development locations is

$$D = \sum_{i=1}^{N} D_{i(j+1)} = \sum_{i=1}^{N} Q'_{ij} \times \mathrm{DF}_{ij}, \tag{13}$$

where j represents the most recently completed testing phase corresponding to each development location.

4. PARAMETER ESTIMATION

In the proposed Type-I and Type-II models, there are four parameters-defect detection ratio σ_{ij}, defect correction ratio θ_{ij}, percentage of added and modified code $\alpha_{i(j+1)}$, and percentage of deleted and modified code $\beta_{i(j+1)}$. Although $\alpha_{i(j+1)}$ and $\beta_{i(j+1)}$ vary depending on different projects, testing phases, and development environments, they can be obtained from the added and modified code $A_{i(j+1)}$, the deleted and modified code $R_{i(j+1)}$, and the original software code size S_{ij} according to the equations derived in Sections 2 and 3. Earlier studies [2] have used the value between 0.86 and 0.94 with the average value around 0.90 for the defect correction ratio θ_{ij}. The defect detection ratio σ_{ij} can be estimated using historical data collected in each distributed development location. With θ_{ij} known, and once DF_{ij} and $D_{i(j+1)}$ are available from the complete defect historical data and code churn information, we can estimate σ_{ij} from equations (2) and (10).

Alternatively, the defect detection ratio σ_{ij} can be estimated based on the published statistics on software defect behavior. Leung [6] first reviews the industry statistics on defects and presents an analysis on defect detection and removal effectiveness under different defect removal phases. Different values are suggested for each software development category, such as military software, system software, commercial software, and MIS. Based on such information and the type of software development category, we obtain an approximate estimate for σ_{ij}.

4.1. Effect of Estimation Parameters

Among the four parameters, defect detection ratio σ_{ij}, defect correction ratio θ_{ij}, percentage of added and modified code $\alpha_{i(j+1)}$, and percentage of deleted and modified code $\beta_{i(j+1)}$, we identify the parameters that contribute most to the variance of the estimated number of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in i^{th} development location. The sensitivity analysis is performed using Monte-Carlo simulation without making any assumption on the possible values for each parameter. The values of the four parameters range from zero to one. Table 1 summarizes the results for both the proposed models. The results show that the defect detection ratio σ_{ij}

Table 1. The effect of estimation parameters on the remaining defects $D_{i(j+1)}$.

Type-I Sequential Debugging Model		
Parameter	Description	Contribution to Variance of $D_{i(j+1)}$
σ_{ij}	Detection Ratio	74.8%
$\alpha_{i(j+1)}$	Percentage of Added Code	17.3%
$\beta_{i(j+1)}$	Percentage of Deleted Code	7.1%
θ_{ij}	Correction Ratio	0.8%
Type-II Iterative Debugging Model		
Parameter	Description	Contribution to Variance of $D_{i(j+1)}$
σ_{ij}	Detection Ratio	78.6%
$\alpha_{i(j+1)}$	Percentage of Added Code	14.6%
$\beta_{i(j+1)}$	Percentage of Deleted Code	6.8%
θ_{ij}	Correction Ratio	0%

has the dominant influence on the variance of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in i^{th} development location.

4.2. Variations in σ_{ij} and θ_{ij}

In this section, the proposed software defect models will be applied to quantitatively estimate the remaining defects with different combinations of defect detection ratio σ_{ij}, defect correction ratio θ_{ij}, percentage of added code $\alpha_{i(j+1)}$, and percentage of deleted code $\beta_{i(j+1)}$ in both Type-I and Type-II models. Consider a case where the percentage of added code $\alpha_{i(j+1)}$ is 20% and the percentage of deleted code $\beta_{i(j+1)}$ is 10%. The defect detection ratio σ_{ij} is varied from 0.2

Table 2. Variation of $D_{i(j+1)}$ in Type-I model and Type-II model for different values of σ_{ij} and θ_{ij}.

	Remaining Defects $D_{i(j+1)}$ in Type-I Model								
θ_{ij}	$\sigma_{ij}=.2$	$\sigma_{ij}=.3$	$\sigma_{ij}=.4$	$\sigma_{ij}=.5$	$\sigma_{ij}=.6$	$\sigma_{ij}=.7$	$\sigma_{ij}=.8$	$\sigma_{ij}=.9$	$\sigma_{ij}=1$
.70	974	911	848	785	722	659	596	533	470
.75	965	898	830	763	695	628	560	493	425
.80	956	884	812	740	668	596	524	452	380
.85	947	871	794	718	641	565	488	412	335
.90	938	857	776	695	614	533	452	371	290
.95	929	844	758	673	587	502	416	331	245
1	920	830	740	650	560	470	380	290	200
	Remaining Defects $D_{i(j+1)}$ in Type-II Model								
θ_{ij}	$\sigma_{ij}=.2$	$\sigma_{ij}=.3$	$\sigma_{ij}=.4$	$\sigma_{ij}=.5$	$\sigma_{ij}=.6$	$\sigma_{ij}=.7$	$\sigma_{ij}=.8$	$\sigma_{ij}=.9$	$\sigma_{ij}=1$
.70	970	902	833	761	688	613	536	456	376
.75	962	890	817	741	665	586	506	424	341
.80	953	878	800	722	642	561	478	394	308
.85	945	865	785	703	620	536	451	365	278
.90	936	853	770	685	599	513	426	338	250
.95	928	842	755	667	579	491	402	313	224
1	920	830	740	650	560	470	380	290	200

Table 3. Variation of $D_{i(j+1)}$ in Remus-Zilles model and Yu-Shen-Dunsmore model for different values of σ_{ij} and θ_{ij}.

	Remaining Defects $D_{i(j+1)}$ in Remus-Zilles Model [1]								
θ_{ij}	$\sigma_{ij} = .2$	$\sigma_{ij} = .3$	$\sigma_{ij} = .4$	$\sigma_{ij} = .5$	$\sigma_{ij} = .6$	$\sigma_{ij} = .7$	$\sigma_{ij} = .8$	$\sigma_{ij} = .9$	$\sigma_{ij} = 1$
.70	946	869	792	715	638	561	484	407	330
.75	935	853	770	688	605	523	440	358	275
.80	924	836	748	660	572	484	396	308	220
.85	913	820	726	633	539	446	352	259	165
.90	902	803	704	605	506	407	308	209	110
.95	891	787	682	578	473	369	264	160	55
1	880	770	660	550	440	330	220	110	0
	Remaining Defects $D_{i(j+1)}$ in Yu-Shen-Dunsmore Model [2]								
θ_{ij}	$\sigma_{ij} = .2$	$\sigma_{ij} = .3$	$\sigma_{ij} = .4$	$\sigma_{ij} = .5$	$\sigma_{ij} = .6$	$\sigma_{ij} = .7$	$\sigma_{ij} = .8$	$\sigma_{ij} = .9$	$\sigma_{ij} = 1$
.70	941	859	774	686	596	504	410	313	215
.75	931	843	754	662	568	472	374	274	172
.80	920	828	734	638	540	441	340	237	132
.85	910	813	715	615	514	411	307	202	95
.90	900	799	696	593	488	383	276	169	61
.95	890	784	678	571	464	356	247	138	29
1	880	770	660	550	440	330	220	110	0

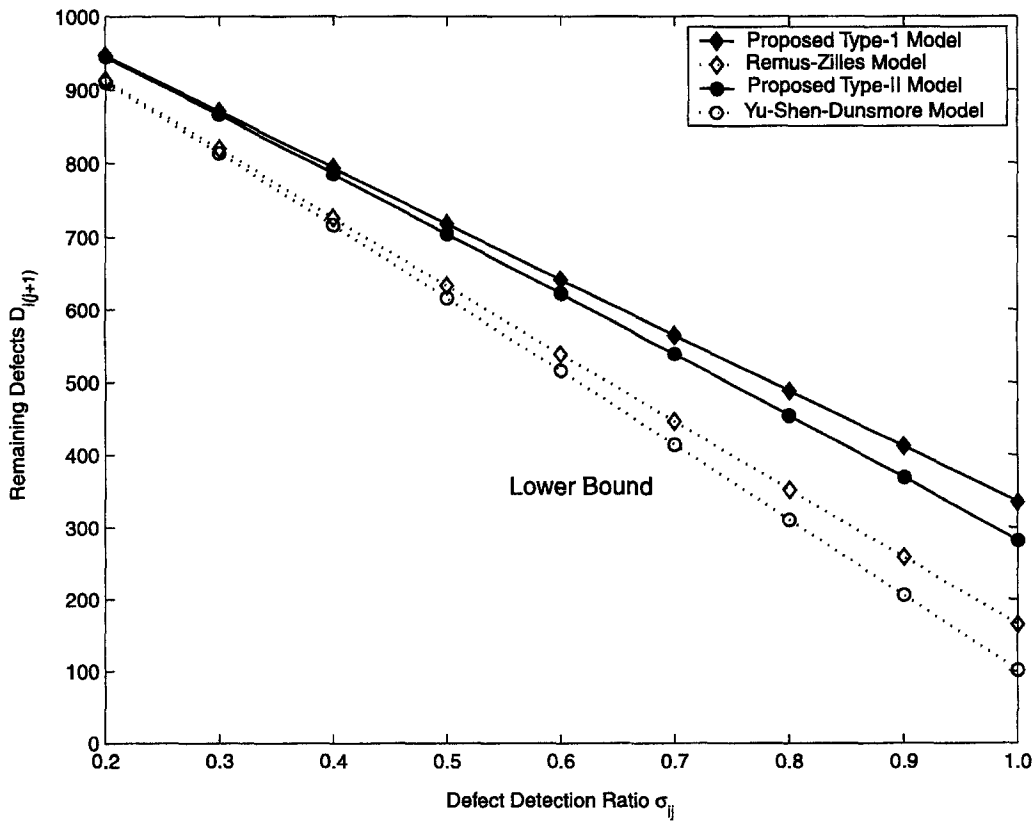

Figure 4. Average number of remaining defects comparisons between our proposed Type-I, Type-II model and the existing approaches in [1,2].

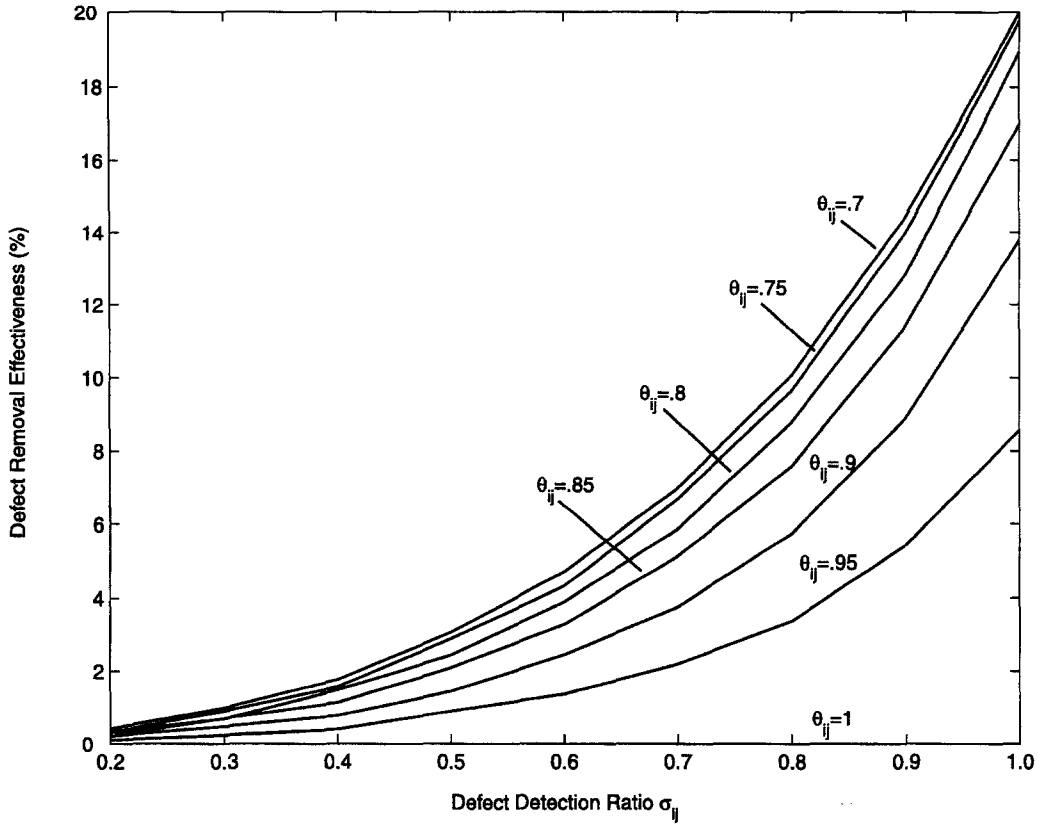

Figure 5. Effectiveness of Type-II model over Type-I model with variations in σ_{ij} and θ_{ij}.

to 1.0, and the defect correction ratio θ_{ij} is varied from 0.7 to 1.0. If the number of original defects D_{ij} is 1000, then the number of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location under different combinations of Type-I and Type-II model are summarized in Table 2.

For comparison purposes, the equivalent number of remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase in both Remus-Zilles model [1] and Yu-Shen-Dunsmore model [2] are summarized in Table 3. Our proposed Type-I model is an extension of the existing Remus-Zilles model [1] with the capability of incorporating the code churn information. The proposed Type-II model is an extension of Yu-Shen-Dunsmore model [2] and also includes the code churn dynamics. Figure 4 compares the two existing approaches with our Type-I and Type-II models. The existing approaches (shown in dotted lines) do not consider the dynamic churn information. This simplistic modeling, therefore, yields a lower bound on estimating the number of remaining defects. Our proposed approaches model real-world software development environment and is able to realistically estimate the remaining defects compared to the existing approaches.

Figure 5 shows the percentage increase in defect removal effectiveness of Type-II iterative debugging model over Type-I sequential debugging model with variations in σ_{ij} and θ_{ij}. Under imperfect debugging conditions where the detection ratio σ_{ij} is high (0.9) and the correction ratio θ_{ij} is low (0.7), the Type-II iterative debugging model removes 14% more defects compared to Type-I sequential debugging model.

4.3. Variations in $\alpha_{i(j+1)}$ and $\beta_{i(j+1)}$

Using the proposed models, we also study the effect of the code churn information on the number of remaining defects. The percentage of added code $\alpha_{i(j+1)}$ and the percentage of deleted code $\beta_{i(j+1)}$ are varied from 0% to 100% for the imperfect debugging case considered earlier.

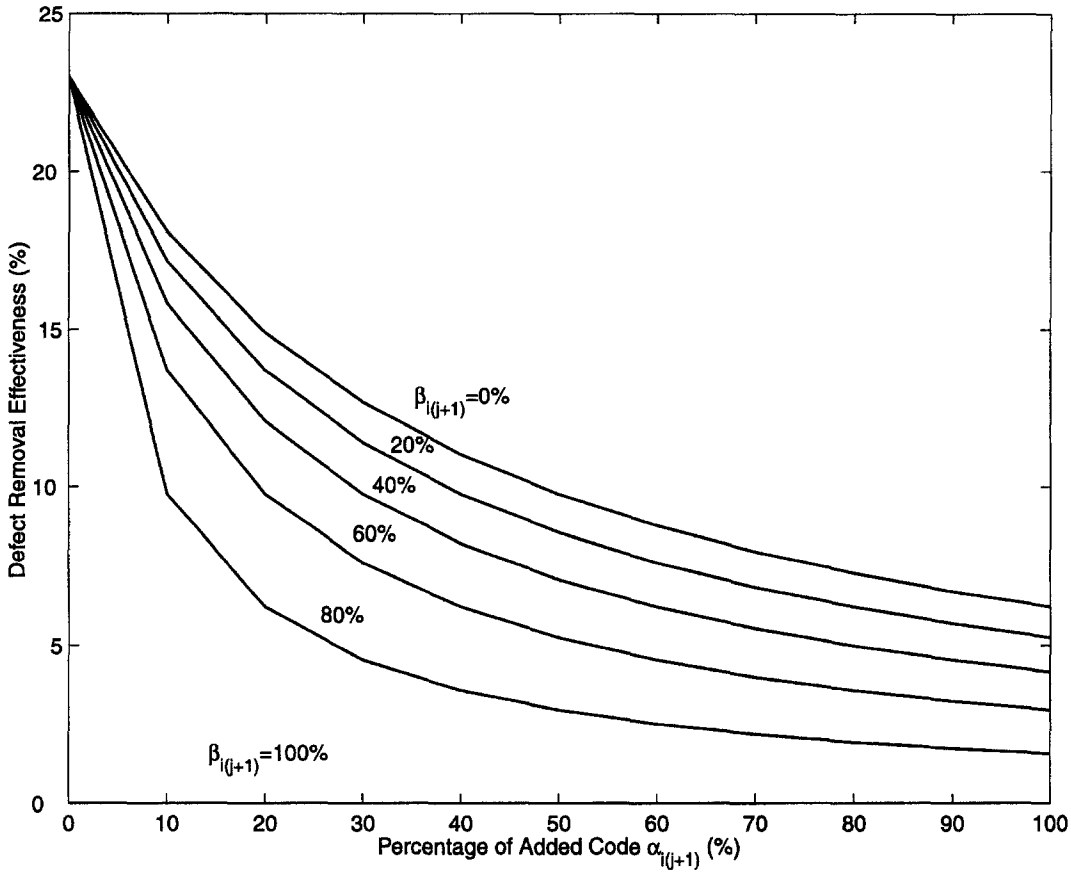

Figure 6. Effectiveness of Type-II model over Type-I model with variations in $\alpha_{i(j+1)}$ and $\beta_{i(j+1)}$.

The number of remaining defects $D_{i(j+1)}$ at the beginning of the $(j+1)^{\text{th}}$ testing phase in the ith development location under different combinations of Type-I and Type-II models can be computed based on equations (2) and (10). Figure 6 shows the percentage increase in defect removal effectiveness of Type-II iterative debugging model over Type-I sequential debugging model with variations in $\alpha_{i(j+1)}$ and $\beta_{i(j+1)}$.

The special case where $\alpha_{i(j+1)}$ is 0% and $\beta_{i(j+1)}$ is 0%, corresponds to the traditional discrete defect removal models [1,2]. The proposed models show that Type-II iterative model is more efficient than Type-I sequential model. For example, when $\alpha_{i(j+1)}$ is 30%, the percentage of deleted code $\beta_{i(j+1)}$ is 20%, the Type-II iterative model is able to remove 11% more defects compared to Type-I sequential debugging model and when $\alpha_{i(j+1)}$ is 20%, the percentage of deleted code $\beta_{i(j+1)}$ is 10%, the improvement is 14%. The model also shows that when the added or deleted codes are small, Type-II model removes more defects than Type-I model. The advantage of using Type-II model over Type-I model becomes less significant when the churn data is very large. In the case of imperfect debugging, the values assigned to parameters σ_{ij} and θ_{ij} can influence the selection of the appropriate debugging model. When the value of σ_{ij} is low, irrespective of the value of θ_{ij}, the efficiency of removing defects is low. This is also observed through the sensitivity analysis performed using Monte-Carlo simulations. When the value of σ_{ij} is high and the value of θ_{ij} is low, then Type-II model is more efficient than Type-I model. Finally, when both σ_{ij} and θ_{ij} are high, there does not exist any significant benefit from using one model over the other.

The proposed Type-II model is more efficient than Type-I model under certain circumstances. Either approach can be used to effectively model the software defect removal process. The choice between the two proposed approaches is dictated by constraints such as the available debugging

time, available resources for debugging, and the software reliability specification for the tolerance threshold of remaining defects in each distributed development location. For example, at the pth development location, if the available debugging time is limited and the defect tolerance threshold is not critical, Type-I model is more suitable. However, if the defect tolerance threshold of the software developed at the qth location is highly critical and the development team has sufficient resources for testing, then Type-II model would be a better choice.

The approaches proposed in this paper are versatile in modeling defects by choosing appropriate values for σ_{ij} and θ_{ij}. These values can be determined based on corporate history, or prior development efforts associated with a given location, or experience of the development team. Our analysis focuses on churn code that is not reusable. However, when the percentage of reusable code in the churn information is known, the values for both σ_{ij} and θ_{ij} will be high and the effective value of $\alpha_{i(j+1)}$ will be lower than the actual value of $\alpha_{i(j+1)}$.

5. CONCLUSION

This paper describes a discrete defect removal model framework that incorporates the dynamics of code churn behavior under a distributed development environment. Two types of defect removal models are derived based on whether the debugging process is sequential or iterative in each testing phase. The mathematical relationship between the number of defects detected during the jth testing phase in different distributed development locations and the total estimated remaining defects at the beginning of the $(j+1)^{\text{th}}$ testing phase for each model is derived.

The detection ratio σ_{ij} is identified to have the greatest contribution to the variance of the estimated number of remaining defects based on the sensitivity analysis using Monte-Carlo simulation. During imperfect debugging, the defect removal effectiveness of Type-I and Type-II models are explored under different values for the parameters such as percentage of added code $\alpha_{i(j+1)}$, percentage of deleted code $\beta_{i(j+1)}$, the defect detection ratio σ_{ij}, and the defect correction ratio θ_{ij}. Compared to the traditional discrete defect removal models, the proposed Type-I and Type-II debugging models are more practical and realistic because they consider the dynamic code churn behavior during software debugging.

REFERENCES

1. H. Remus and S.N. Zilles, Prediction and management of program quality, In *Proceedings of the Fourth International Conference on Software Engineering*, pp. 341–350, Munich, Germany, (1979).
2. T.J. Yu, V.Y. Shen and H.E. Dunsmore, An analysis of several software defect models, *IEEE Trans. Software Eng.* **14** (9), 1261–1270, (1988).
3. M. Coleman and J. Allan, Defect removal models: Theory and practice, In *Proceedings of Software Engineering*, pp. 372–384, Brighton, U.K., (1990).
4. S.H. Kan, Modeling and software development quality, *IBM Systems Journal* **30** (3), 351–362, (1991).
5. S.H. Kan, *Metrics and Models in Software Quality Engineering, Chapter 6, Defect Removal Effectiveness*, pp. 159–185, Addison-Wesley, (2003).
6. H.K.N. Leung, Improving defect removal effectiveness for software development, In *Proceedings of the Second Euromicro Conference on Software Maintenance and Reengineering*, pp. 157–164, Florence, Italy, (1998).
7. K.Y. Cai, *Software Defect and Operational Profile Modeling, Chapter 7, Software Defect Estimations Under Imperfect Debugging*, pp. 163–186, The Kluwer International Series in Software Engineering, Kluwer Academic, (1998).
8. N. Karunanithi, A neural network approach for software reliability growth modeling in the presence of code churn, In *Proceedings of the Fourth International Symposium on Software Reliability Engineering*, pp. 310–317, Denver, CO, (1993).
9. J.C. Munson and S.G. Elbaum, Code churn: A measure for estimating the impact of code change, In *Proceedings of International Conference on Software Maintenance*, pp. 24–31, Bethesda, MD, (1998).
10. G.A. Hall and J.C. Munson, Software evolution: Code delta and code churn, *The Journal of Systems and Software* **54** (2), 111–118, (2000).